

# Tuning and Benchmarking of RocksDB

An Empirical Study of Parameter Sensitivity in RocksDB

**Meet Vaghasiya**

Student ID: 202511020

MTech Major Project 1 (PC876)

*Under the guidance of*

**Prof. P. M. Jat**

Dhirubhai Ambani University, Gandhinagar

# Problem Statement & Motivation

## The Problem:

- RocksDB deployments use **static, default** configurations set at database initialization
- Configuration optimized for writes slows read performance and vice versa
- Selecting the optimal configuration remains a difficult engineering challenge
- No published study systematically quantifies the **sensitivity** of each tuning knob across data scales [3,4]

## Our Objective

- Quantify the **individual impact** of six RocksDB tuning knobs on throughput, latency, DB size, and compaction
- Validate whether **parameter rankings remain stable** from 8M to 100M records
- Derive a **recommended starting configuration** for production deployments

# Background: LSM Tree Architecture

## Core Components:

### 1 Write-Ahead Log (WAL)

- ▷ Records every write before applying
- ▷ Ensures durability on system crash

### 2 MemTable (In-Memory Buffer)

- ▷ Sorted buffer (SkipList) for fast writes [1]
- ▷ Flushed to disk as SSTable when full

### 3 SSTables (Sorted String Tables)

- ▷ Immutable, sorted files organized in levels [1]
- ▷ L0 allows key overlap, L1+ does not [5]

### 4 Compaction

- ▷ Merges SSTables across levels [1]
- ▷ Removes stale data and tombstones [5]

## Write Pipeline

- 1 Write arrives → recorded in WAL
- 2 Inserted into active MemTable
- 3 MemTable fills → becomes immutable
- 4 Flushed to L0 as sorted SSTable
- 5 Background compaction merges levels [7]

# Experimental Setup

## System Environment

| Component | Specification           |
|-----------|-------------------------|
| Platform  | WSL2 on Windows (drvfs) |
| CPU       | Intel i5-10505 3.20 GHz |
| Cores     | 6 Cores, 12 Threads     |
| RAM       | 16 GB DDR4              |
| Storage   | 256 GB SSD NVMe         |
| RocksDB   | v11.6.0 (from source)   |

## Metrics Measured

Throughput, Read latency (p99), DB size, Flush count, Compaction time, Stall time

## Benchmark Tools & Workloads

| Tool     | Phase      | Detail                |
|----------|------------|-----------------------|
| db_bench | Technique  | readrandomwriterandom |
|          | RW percent | 50 - 50               |
|          | Data scale | 8/10/50/100M records  |

**db\_bench:** 1 KB value size, 16 threads

## Assumption

WSL2 drvfs storage relies on OS page cache.

# Baseline Configuration & Parameter Space

## Baseline (Default values)

| Parameter           | Default |
|---------------------|---------|
| compression         | lz4     |
| write_buffer_size   | 64 MB   |
| max_background_jobs | 4       |
| block_cache         | 1 GB    |
| use_direct_reads    | false   |
| compaction_style    | level   |
| value_size          | 1 KB    |
| threads             | 16      |

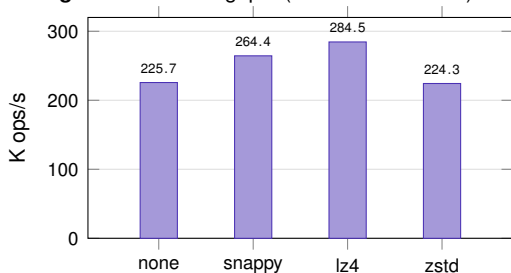
## Parameter Space

| Knob             | Values Tested           |
|------------------|-------------------------|
| Compression      | none, snappy, lz4, zstd |
| MemTable size    | 64, 128, 256, 512 MB    |
| Background jobs  | 2, 4, 8, 16             |
| Block cache      | 512 MB, 1, 2, 4 GB      |
| Direct I/O       | OFF, ON                 |
| Compaction style | Level, Universal, FIFO  |

Methodology: single parameter change at a time

# Result: Compression Types

**Fig. 1: Mean Throughput (8M-100M Records)**



Mean across 4 scales (8M, 10M, 50M, 100M records). LZ4 achieves peak throughput and smallest DB size

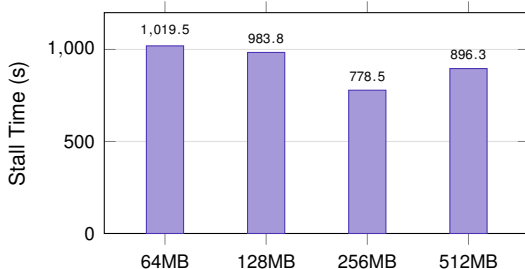
| type       | K ops/s      | DB MB        | p99 $\mu$ s |
|------------|--------------|--------------|-------------|
| none       | 225.7        | 7,450        | 23.2        |
| snappy     | 264.4        | 4,462        | 19.8        |
| <b>lz4</b> | <b>284.5</b> | <b>3,854</b> | <b>18.0</b> |
| zstd       | 224.3        | 4,783        | 23.2        |

## Key Findings:

- **LZ4** out performed with +26% throughput and 48% smaller DB size than uncompressed
- **Read Latency:** Smaller blocks maximize OS page cache, making reads fast.
- **Amplification:** 48% smaller data size which reduce compaction write amplification.
- Use of compression techniques like lz4 or snappy for this workload is good choice

# Result: MemTable Size (write\_buffer\_size)

**Fig. 2:** Compaction Stall Time by MemTable Size



Stall time in seconds across 4 scales (8M, 10M, 50M, 100M records). 256 MB achieves the lowest stall time, balancing flush frequency and compaction overhead

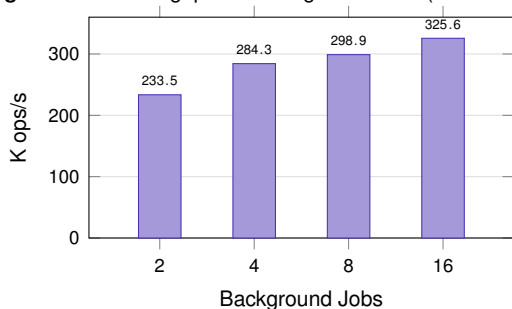
| Size          | K ops/s      | Flush      | DB MB        | Stall (s)    |
|---------------|--------------|------------|--------------|--------------|
| 64 MB         | 257.1        | 672        | 4,410        | 1,019.5      |
| 128 MB        | 253.5        | 336        | 5,218        | 983.8        |
| <b>256 MB</b> | <b>269.3</b> | <b>163</b> | <b>4,431</b> | <b>778.5</b> |
| 512 MB        | 288.2        | 84         | 9,376        | 896.3        |

## Key Findings:

- **256 MB** buffer size gives balanced throughput (269K ops/s) while achieving the lowest compaction stall time and database size.
- **Amplification at 512 MB:** Maximizes throughput by minimizing flush I/O, but triggers severe space amplification due to delayed compactions.
- **Predictable I/O Reduction:** Flush counts nearly halve exactly as buffer size doubles, directly validating that larger MemTables reduce disk write frequency.

# Result: Background Jobs (max\_background\_jobs)

**Fig. 3:** Mean Throughput vs Background Jobs (8M-100M)



Monotonic gain +39% throughput from 2 to 16 jobs, consistent across all scales

| Jobs | K ops/s | Gain | p99 $\mu$ s |
|------|---------|------|-------------|
| 2    | 233.5   | -    | 22.2        |
| 4    | 284.3   | +22% | 18.0        |
| 8    | 298.9   | +28% | 17.5        |
| 16   | 325.6   | +39% | 16.0        |

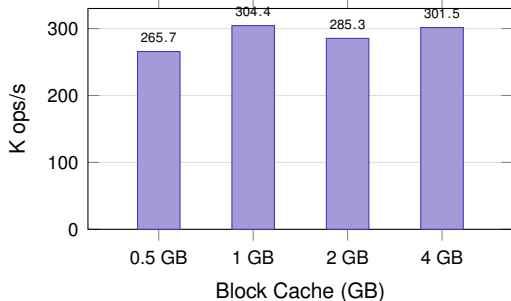
## Key Findings:

- **Scaling** 2→16 threads gave identical gains at all dataset sizes
- **Latency improves:** P99 drops 28% (22.2 to 16.0  $\mu$ s) with zero space penalty
- **Throughput gain** collapses for higher number of threads, because thread management overhead increases



# Result: Block Cache Size

**Fig. 4:** Mean Throughput vs Block Cache Size (8M-100M)



1 GB Block Cache effectively holding working set and fits in cache

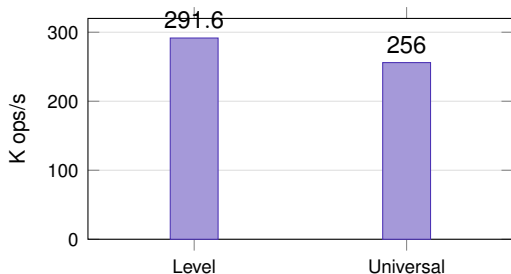
| Cache       | K ops/s      | p99 $\mu$ s | Effects          |
|-------------|--------------|-------------|------------------|
| 512 MB      | 265.7        | 20.5        | Cache misses     |
| <b>1 GB</b> | <b>304.4</b> | <b>17.8</b> | <b>Optimal</b>   |
| 2 GB        | 285.3        | 18.8        | Small OS cache   |
| 4 GB        | 301.5        | 18.2        | Smaller OS cache |

## Key Findings:

- **1 GB is saturation point:** Further allocation do not contribute to throughput result
- **512 MB under provisioned:** 14% throughput lower, because of more cache misses
- **Over allocation is issue:** 2 GB shows 6.3% degradation in throughput overhead of block cahce management on RAM
- Size cache to working set size. Allocate remaining RAM to memtable/OS page cache

# Result: Compaction Style

**Fig. 5:** Mean Throughput by Compaction Style  
(8M-100M)



## FIFO Not Tested for Durability

FIFO is suitable only for time window data (logs, metrics) where old keys auto expire. Excluded from this comparison.

| Style     | K ops/s | DB MB | p99 $\mu$ s |
|-----------|---------|-------|-------------|
| Level     | 291.6   | 4,515 | 17.8        |
| Universal | 256.0   | 8,836 | 20.2        |

## Key Findings:

- **Level compaction** better performs with +14% throughput with 48% smaller DB size and 12% lower P99 latency
- **Universal had 2x space amplification**, because of it's delayed compaction and cache inefficiency
- **Level compaction** is better for mixed workload. Universal for write intensive, where DB size is not an issue
- **Level's active compaction** keeps minimal overlapping sorted runs per level keep reads and space amplification low

## Recommended Configuration

| Parameter           | Value         | Rationale                                             |
|---------------------|---------------|-------------------------------------------------------|
| compression         | <b>lz4</b>    | small DB size + fast reads (40% space saving)         |
| write_buffer_size   | <b>256 MB</b> | fewer flushes, lowest stalls, limited DB size         |
| max_background_jobs | <b>8-16</b>   | Scale this upto CPU threads                           |
| block_cache         | <b>≈ WS</b>   | Size of working set.                                  |
| compaction_style    | <b>level</b>  | Fast reads + low space amplification + mixed workload |

## Workload Aware Decision Guide

- **Mixed workload:** lz4 compression + Level compaction + 256 MB memtable + 8 bg jobs

# Limitations & Future Work

## Limitations

- **Environment:** All results from WSL2 drvfs
- **Single parameter at a time:** Cross parameter interactions were not captured
- **Synthetic workload:** db\_bench uses synthetic workload generation not completely realistic
- **Fixed value size** (1 KB): Results may differ for higher value size

## Future Work

- **Testing parameters Together:** Test multiple parameters at the same time to see how they interact with each other.
- **Running on different environment:** Repeat experiments on dedicated NVMe SSDs
- **Larger value sizes:** Test 4 KB, 16 KB, 64 KB to validate compression recommendations

# References I

- ❶ P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The Log-Structured Merge-Tree (LSM-Tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351-385, 1996.
- ❷ S. Dong, M. Callaghan, L. Galanis, D. Borthakur, T. Savor, and M. Strum, "Optimizing Space Amplification in RocksDB," in *Proc. CIDR*, Jan. 2017.
- ❸ C. Luo and M. J. Carey, "LSM-based Storage Techniques: A Survey," *The VLDB Journal*, vol. 29, no. 1, pp. 393-418, 2020.
- ❹ S. Sarkar and M. Athanassoulis, "Dissecting, Designing, and Optimizing LSM-based Data Stores," in *Proc. ACM SIGMOD '22*, Jun. 2022.
- ❺ S. Dong, A. Kryczka, Y. Jin, and M. Stumm, "RocksDB: Evolution of Development Priorities in a Key-Value Store Serving Large-scale Applications," *ACM Trans. Storage*, vol. 17, no. 4, Article 26, Oct. 2021.
- ❻ B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking Cloud Serving Systems with YCSB," in *Proc. ACM SoCC '10*, 2010.
- ❼ Facebook/RocksDB, *RocksDB Wiki*, GitHub.  
<https://github.com/facebook/rocksdb/wiki>

# **Thank You**

**Meet Vaghasiya**

Student ID: 202511020

Dhirubhai Ambani University, Gandhinagar